Plantilla de Informe de Trabajo Práctico

1. Carátula

- **Título del trabajo:** TRABAJO PRÁCTICO Nº 2: Aplicaciones de estructuras jerárquicas y grafos
- **Integrantes del equipo:**
  - Bautista Kadur El Ainie
  - Fausto Siviero
- **Fecha de entrega:** 02/11/2026
- **Link del repositorio:** https://github.com/FaberCasio/AyED2026c2-Briggiler-Kadur-El-Ainie-Siviero.git

2. Introducción

El trabajo práctico aplica montículos, árboles AVL y grafos a tres problemas donde la elección de la estructura de datos determina el costo de las operaciones críticas.

Los problemas resueltos son:

- **Proyecto 1, Sala de Emergencias:** atender siempre al paciente de riesgo más delicado, con un segundo criterio de desempate, mediante una cola de prioridad genérica implementada sobre un montículo binario.
- **Proyecto 2, Temperaturas_DB:** guardar mediciones de temperatura asociadas a una fecha y consultarlas por rango de forma eficiente, mediante un árbol AVL.
- **Proyecto 3, Servicio de Transporte:** hallar el peso máximo transportable (máximo cuello de botella) y el precio mínimo de transporte desde Buenos Aires hacia cada ciudad, mediante un grafo dirigido y dos variantes del algoritmo de Dijkstra que reutilizan la cola de prioridad del Proyecto 1 (ajustada a máximos).

Los temas teóricos aplicados son:

- Tipos de datos abstractos y separación entre implementación y aplicación.
- Montículo binario y cola de prioridad (de mínimos y de máximos).
- Árboles binarios de búsqueda balanceados (AVL) y sus rotaciones.
- Grafos ponderados y el algoritmo de Dijkstra.
- Análisis de complejidad temporal con notación O-grande.

3. Desarrollo de los problemas

3.1. Problema Nº 1: Sala de Emergencias

A. Análisis y planteo

El problema consiste en administrar una colección de pacientes en espera donde el próximo en salir no es el que llegó primero sino el de riesgo más delicado. Las operaciones críticas son insertar un paciente cuando ingresa y extraer el de mayor urgencia cuando hay atención.

Con una lista simple, o una lista ordenada, una de las dos operaciones cuesta O(n): buscar el más urgente recorre toda la lista, y mantenerla ordenada obliga a desplazar elementos al insertar. Usamos una **cola de prioridad implementada sobre un montículo binario de mínimos**, donde ambas operaciones cuestan O(log n). Como el nivel de riesgo 1 es el crítico, el menor número tiene la mayor prioridad, y por eso corresponde un montículo de mínimos.

El criterio de desempate para pacientes con el mismo nivel de riesgo es el **orden de llegada** (FIFO). La elección es razonable cuando la cantidad de pacientes en espera puede crecer: la ventaja frente a una lista aparece al aumentar n. La estructura es **genérica**: guarda cualquier elemento junto con una prioridad y no conoce la clase `Paciente`, lo que separa la implementación de la aplicación.

B. Implementación / Resolución

Se implementaron dos clases:

- **`MonticuloBinario`:** arreglo con el índice 0 como marcador, de modo que el hijo izquierdo de `i` está en `2i`, el derecho en `2i+1` y el padre en `i // 2`. Sus métodos son `insertar`, `eliminarMin`, `infiltArriba`, `infiltAbajo` e `hijoMin`.
- **`ColaDePrioridad`:** guarda en el montículo tuplas `(prioridad, contador_llegada, elemento)`. Python compara las tuplas campo por campo, así que ante igual prioridad desempata el contador. Como el contador es único, nunca se comparan los elementos entre sí, y por eso la cola acepta cualquier tipo de dato.

En `main.py` la lista simple de la simulación se reemplazó por la cola de prioridad. La prioridad es el nivel de riesgo del paciente:

```python
cola_de_espera = ColaDePrioridad()
```

```
cola_de_espera.insertar(paciente,  paciente.get_riesgo())      # ingreso

paciente_atendido = cola_de_espera.eliminar_min()              # atención
```

C. Resultados y pruebas

Se ejecutó la simulación durante 20 ciclos (un paciente nuevo por ciclo, con atención en aproximadamente la mitad de ellos). La tabla resume lo observado en la salida:

Hora	Paciente atendido	Interpretación
11:05:20	Leandro Lopez (1-crítico)	Pasó delante de un moderado y de dos pacientes de riesgo bajo que habían llegado antes
11:05:22	Andrea Belgrano (2-moderado)	Con el crítico ya atendido, el moderado sale antes que los pacientes de riesgo bajo
11:05:24 y 11:05:25	Antonio Juarez y Andrea Mendez (3-bajo)	Entre pacientes del mismo riesgo se respeta el orden de llegada (11:05:17 y 11:05:19)
11:05:26	Gastón Belgrano (1-crítico)	Llegó en ese mismo ciclo y se atendió de inmediato, delante de 6 pacientes de riesgo bajo
11:05:30	Mariela García (2-moderado)	Se atendió de inmediato, delante de 9 pacientes de riesgo bajo

11:05:31 a 11:05:35	Mariela Rodriguez, Gastón Colman, Estela Belgrano, Gastón Rodriguez y Andrea García (3-bajo)	Con solo pacientes de riesgo bajo en espera, salen en orden de llegada (11:05:20 a 11:05:24)

El listado "Pacientes que faltan atenderse" recorre el arreglo interno del montículo, por lo que no se muestra en orden de atención. La estructura solo garantiza que la raíz sea el próximo paciente.

```
 02/11/2026 22:15:05

****************************************
Se atiende el paciente: Leandro Mendez   -> 1-crítico
****************************************

Pacientes que faltan atenderse: 2
        Agustina Colman         -> 3-bajo
        Agustina Belgrano       -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:06

****************************************
Se atiende el paciente: Estela Colman    -> 2-moderado
****************************************

Pacientes que faltan atenderse: 2
        Agustina Colman         -> 3-bajo
        Agustina Belgrano       -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:07

****************************************
Se atiende el paciente: Agustina Colman  -> 3-bajo
****************************************

Pacientes que faltan atenderse: 2
        Agustina Belgrano       -> 3-bajo
        Andrea Belgrano         -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:08

****************************************
Se atiende el paciente: Agustina Belgrano        -> 3-bajo
****************************************

Pacientes que faltan atenderse: 2
        Andrea Belgrano         -> 3-bajo
        Andrea Mendez   -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
```

```
 02/11/2026 22:15:09


Pacientes que faltan atenderse: 3
        Andrea Belgrano         -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:10

****************************************
Se atiende el paciente: Andrea Belgrano  -> 3-ba
****************************************

Pacientes que faltan atenderse: 3
        Andrea Mendez   -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Jorge Colman    -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:11


Pacientes que faltan atenderse: 4
        Jorge Rodriguez         -> 1-crítico
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
        Andrea Lopez    -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

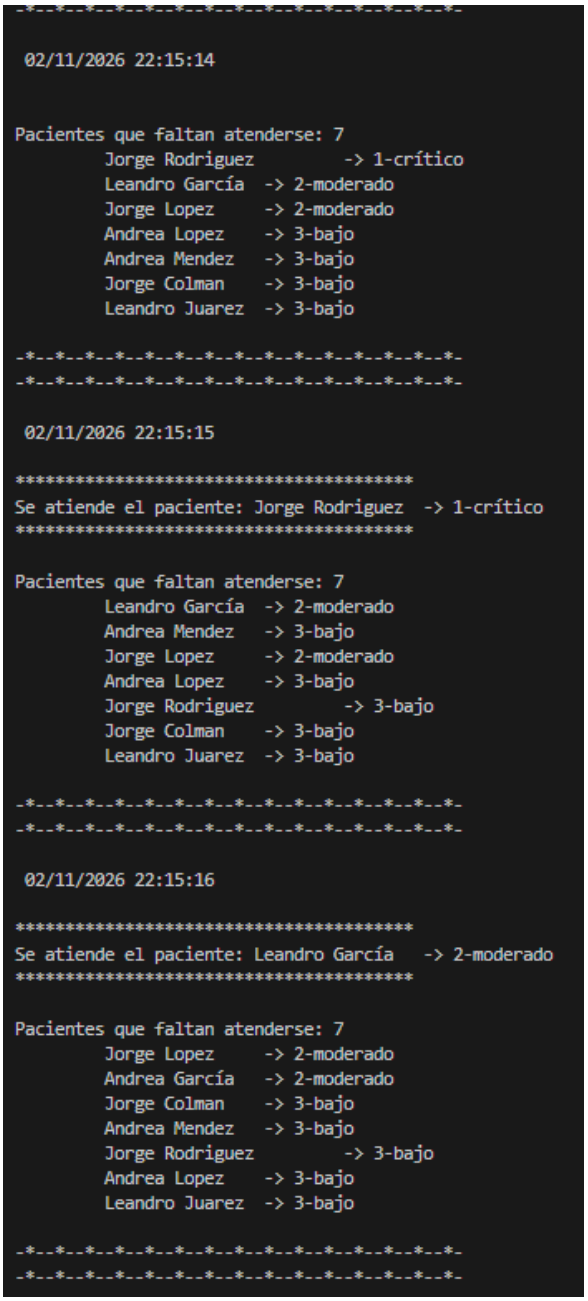
 02/11/2026 22:15:12


Pacientes que faltan atenderse: 5
        Jorge Rodriguez         -> 1-crítico
        Leandro García  -> 2-moderado
        Jorge Colman    -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Andrea Mendez   -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:13


Pacientes que faltan atenderse: 6
        Jorge Rodriguez         -> 1-crítico
        Leandro García  -> 2-moderado
        Jorge Lopez     -> 2-moderado
        Andrea Lopez    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
```



```
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:14


Pacientes que faltan atenderse: 7
        Jorge Rodriguez         -> 1-crítico
        Leandro García  -> 2-moderado
        Jorge Lopez     -> 2-moderado
        Andrea Lopez    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
        Leandro Juarez  -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:15

**************************************
Se atiende el paciente: Jorge Rodriguez  -> 1-crítico
**************************************

Pacientes que faltan atenderse: 7
        Leandro García  -> 2-moderado
        Andrea Mendez   -> 3-bajo
        Jorge Lopez     -> 2-moderado
        Andrea Lopez    -> 3-bajo
        Jorge Rodriguez         -> 3-bajo
        Jorge Colman    -> 3-bajo
        Leandro Juarez  -> 3-bajo

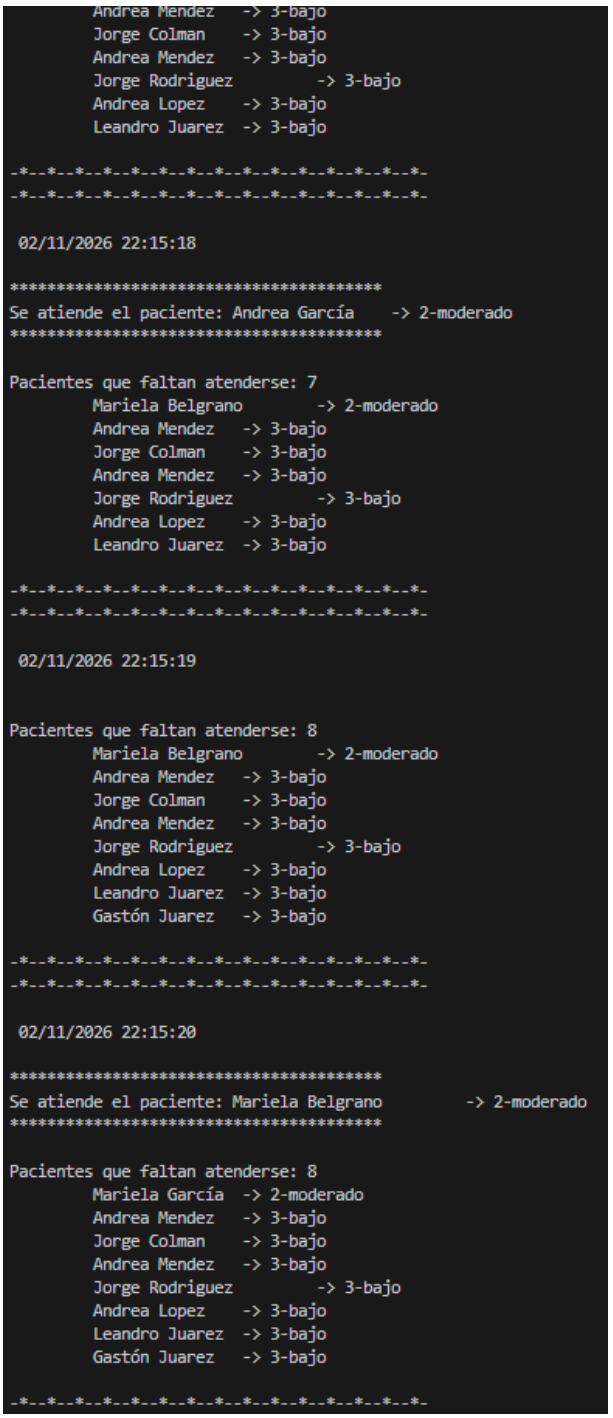
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:16

**************************************
Se atiende el paciente: Leandro García   -> 2-moderado
**************************************

Pacientes que faltan atenderse: 7
        Jorge Lopez     -> 2-moderado
        Andrea García   -> 2-moderado
        Jorge Colman    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Rodriguez         -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Leandro Juarez  -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
```



```
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Rodriguez         -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Leandro Juarez  -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:18

**************************************
Se atiende el paciente: Andrea García    -> 2-moderado
**************************************

Pacientes que faltan atenderse: 7
        Mariela Belgrano        -> 2-moderado
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Rodriguez         -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Leandro Juarez  -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:19


Pacientes que faltan atenderse: 8
        Mariela Belgrano        -> 2-moderado
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Rodriguez         -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Leandro Juarez  -> 3-bajo
        Gastón Juarez   -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-

 02/11/2026 22:15:20

**************************************
Se atiende el paciente: Mariela Belgrano         -> 2-moderado
**************************************

Pacientes que faltan atenderse: 8
        Mariela García  -> 2-moderado
        Andrea Mendez   -> 3-bajo
        Jorge Colman    -> 3-bajo
        Andrea Mendez   -> 3-bajo
        Jorge Rodriguez         -> 3-bajo
        Andrea Lopez    -> 3-bajo
        Leandro Juarez  -> 3-bajo
        Gastón Juarez   -> 3-bajo

-*--*--*--*--*--*--*--*--*--*--*--*--*--*--*-
```

3.2. Problema Nº 2: Temperaturas_DB

A. Análisis y planteo

El problema consiste en mantener en memoria una base de datos de mediciones de temperatura indexadas por fecha, que crece con el tiempo y se consulta sobre todo por

rangos de fechas. Las operaciones críticas son insertar una medición, buscarla por fecha, borrarla y recorrer un rango de fechas para obtener el máximo, el mínimo o el listado ordenado.

Se eligió un **árbol AVL** con la fecha como clave y la temperatura como valor. Las alternativas resuelven bien solo una parte: una lista desordenada inserta en O(1) pero busca y consulta rangos en O(n), y una lista ordenada busca en O(log n) pero inserta en O(n). El AVL mantiene las claves ordenadas y se autobalancea, así que insertar, buscar y borrar cuestan O(log n), y un rango se obtiene recorriendo solo la parte del árbol que lo cubre.

El balanceo importa en este caso. El archivo de muestras viene ordenado cronológicamente, y un árbol binario de búsqueda sin balancear que recibe claves ordenadas degenera en una lista de altura n. La elección es razonable cuando el conjunto de mediciones crece y las consultas por rango son frecuentes. Se asume una sola medición por fecha: si se guarda de nuevo una fecha existente, se actualiza su temperatura.

B. Implementación / Resolución

Se implementaron dos clases:

- **`ArbolAVL` y `NodoArbolAVL`:** árbol genérico de pares clave-valor. Cada nodo guarda su altura y su factor de equilibrio, y tras insertar o eliminar se aplican las cuatro rotaciones (simple derecha, simple izquierda, doble izquierda-derecha y doble derecha-izquierda). El método `inorden_con_poda(min, max)` recorre solo las ramas que pueden contener claves dentro del rango.
- **`TemperaturasDB`:** usa un `ArbolAVL` interno y ofrece la interfaz pedida. Convierte la fecha `dd/mm/aaaa` a un objeto `datetime.date`, de modo que el árbol compara fechas y no texto. Valida que `fecha1 < fecha2`, pero los extremos del rango no tienen que existir en el árbol.

Las consultas de rango delegan en `inorden_con_poda` y luego calculan el máximo, el mínimo o el listado sobre los valores obtenidos:

Python

```python
def max_temp_rango(self, fecha1, fecha2):

    d1, d2 = self.__validar_rango(fecha1, fecha2)

    datos = self.__arbol.inorden_con_poda(d1, d2)

    if not datos:
```

```
        raise ValueError(f"No hay datos en el rango '{fecha1}' -
'{fecha2}'")

    return max(dato[1] for dato in datos)
```

Para cargar datos se implementó `cargar_desde_archivo(ruta)`. Lee líneas con el formato `dd/mm/aaaa;temperatura`, ignora con una advertencia las líneas inválidas y devuelve la cantidad de líneas cargadas.

C. Resultados y pruebas

La clase `ArbolAVL` se probó con una suite de `unittest` de 9 pruebas: árbol vacío, las cuatro rotaciones, búsqueda y sobrescritura, eliminación de la única raíz, eliminación de un nodo con dos hijos y una prueba de estrés con 500 claves aleatorias que incluye recorrido con poda y eliminación de 450 claves. Después de cada caso, un método verificador comprueba que las claves estén ordenadas y sin repetir, que el tamaño sea consistente y que la altura no supere la cota teórica de un AVL. El resultado fue `Ran 9 tests ... OK`.

Luego se cargó el archivo de muestras, con 120 mediciones del 01/01/2025 al 30/04/2025, y se probaron las operaciones:

Operación	Resultado
`cargar_desde_archivo` y `cantidad_muestras()`	120 líneas leídas, 120 muestras
`devolver_temperatura("15/03/2025")`	26.7
`max_temp_rango("01/01/2025", "31/01/2025")`	39.5 (11/01)

`min_temp_rango("01/01/2025", "31/01/2025")`	11.9 (28/01)
`temp_extremos_rango("01/02/2025", "28/02/2025")`	(14.8, 39.9)
`temp_extremos_rango("31/12/2024", "05/01/2025")`	(13.2, 38.6): el extremo inicial no existe en la base y el rango igual se resuelve
`devolver_temperaturas("01/03/2025", "05/03/2025")`	5 líneas ordenadas, de `01/03/2025: 21.8 °C` a `05/03/2025: 24.3 °C`
`borrar_temperatura("15/03/2025")`	La cantidad pasa de 120 a 119 y consultar esa fecha lanza `ValueError`
`max_temp_rango("10/01/2025", "05/01/2025")`	`ValueError`: la fecha inicial debe ser anterior a la final

Con las 120 muestras, que se insertaron en orden cronológico, la altura del árbol resultó 7. Es la mínima posible para 120 nodos, ya que un árbol de altura 6 admite como máximo 63. Esto confirma que el balanceo evita la degeneración en lista.

salida del test para el árbol AVL:

```
.........
----------------------------------------------------------------------
Ran 9 tests in 0.019s

OK
```

main de ejemplo para corroborar Temperaturas_DB:

```
líneas leídas: 120 | muestras: 120
15/03/2025: 26.7
máx enero: 39.5
mín enero: 11.9
extremos febrero: (14.8, 39.9)
extremos con fecha inicial inexistente: (13.2, 38.6)
   01/03/2025: 21.8 ºC
   02/03/2025: 37.0 ºC
   03/03/2025: 32.4 ºC
   04/03/2025: 27.3 ºC
   05/03/2025: 24.3 ºC
tras borrar: 119
```

3.3. Problema Nº 3: Servicio de Transporte

A. Análisis y planteo

El problema consiste en recorrer una red de ciudades unidas por rutas de un solo sentido, cada una con dos datos: una capacidad en kg y un precio. Se pide responder dos preguntas desde Buenos Aires hacia cada ciudad:

1. **Ítem 1, peso máximo W:** es el *camino más amplio*. El valor de un camino es la menor capacidad de sus rutas (el cuello de botella) y se busca el camino que maximiza ese valor.
2. **Ítem 2, precio mínimo para transportar W:** es un camino más barato (suma de precios), usando solo las rutas con capacidad mayor o igual a W.

Las operaciones críticas son extraer el vértice con mejor valor provisional, recorrer sus vecinos y actualizar el valor de un vértice ya cargado en la cola. Se eligió representar la red con un **grafo dirigido con listas de adyacencia** (clases `Vertice` y `Grafo`) y resolver ambos ítems con el **algoritmo de Dijkstra** apoyado en la cola de prioridad del Proyecto 1 (modificada).

El enfoque de Dijkstra es válido en ambos ítems por el mismo motivo: al extender un camino, el precio acumulado nunca baja y el cuello de botella nunca sube. Por eso, el primer vértice que sale de la cola ya tiene su valor definitivo. El supuesto es que capacidades y precios son enteros positivos, como indica el enunciado.

B. Implementación / Resolución

Cada `Vertice` guarda una sola ponderación por arista. Por eso se arman **dos grafos** a partir de las mismas rutas: uno con la capacidad como ponderación (ítem 1) y otro con el

precio, que incluye solo las rutas con capacidad mayor o igual a W (ítem 2). El archivo de rutas se carga en el sentido en que está escrito, ya que una ruta de A a B no implica la de B a A.

Para el ítem 1 fue necesario pasar la cola de prioridad a **máximos**: se agregaron `MonticuloMaximos` y `ColaDePrioridadMax`, con las comparaciones invertidas y el método `aumentar_clave`. En la cola de máximos el contador de llegada se guarda negado para que los empates sigan saliendo por orden de llegada.

`dijkstra_mod` es el Dijkstra clásico con los signos cambiados:

	`dijkstra` **(precio mínimo)**	**`dijkstra_mod`** **(peso máximo)**
Valor inicial del origen	0	infinito
Valor inicial de los demás	infinito	0
Cola de prioridad	`ColaDePrioridad` (mínimos)	`ColaDePrioridadMax` (máximos)
Valor al pasar por una arista	`distancia + ponderación`	`min(distancia, ponderación)`
Se actualiza el vecino si	el nuevo valor es menor	el nuevo valor es mayor
Actualización en la cola	`decrementar_clave`	`aumentar_clave`

El programa principal sigue los siguientes pasos: carga las rutas, arma el grafo de capacidades con todas las distancias en 0 y ejecuta `dijkstra_mod`; luego toma el W del destino elegido, arma el grafo de precios filtrado por W y ejecuta `dijkstra`. El camino se reconstruye siguiendo `obtenerPredecesor()` desde el destino.

C. Resultados y pruebas

Se ejecutó el programa con el archivo `rutas.txt` (origen: CiudadBs.As.). El ítem 1 devolvió el peso máximo hacia las 24 ciudades de destino. Para dos destinos se verificó además el ítem 2:

Destino	W máximo (kg)	Precio mínimo	Camino
Salta	70	33	CiudadBs.As. → Rosario → Rafaela → S.delEstero → SanMiguelTucuman → Salta
Mendoza	80	29	CiudadBs.As. → Rufino → Laboulage → VillaMercedes → SanLuisCap → Mendoza

El caso de Salta se comprobó a mano. El cuello de botella es 70 (ruta Rosario → Rafaela) y el precio es 7 + 9 + 8 + 3 + 6 = 33.

Mendoza es un caso donde el filtro por capacidad cambia el resultado. Sin filtro, el camino más barato pasa por RioCuarto y cuesta 27, pero la ruta RioCuarto → SanLuisCap admite solo 60 kg y el W de Mendoza es 80. Al descartarla, el camino válido más barato cuesta 29.

```
Peso máximo desde CiudadBs.As.:
  BellaVista: 50
  CiudadCordoba: 50
  Frias: 50
  Gral.Guemes: 70
  LaRioja: 70
  Laboulage: 90
  MarcosJuarez: 80
  Mendoza: 80
  MinaClavero: 50
  Rafaela: 70
  RioCuarto: 90
  Rosario: 90
  RosarioFrontera: 70
  Rufino: 100
  S.F.ValleCatamarca: 70
  S.delEstero: 70
  Salta: 70
  SanLuisCap: 80
  SanMiguelTucuman: 70
  SanSalvadorJujuy: 70
  Simoca: 50
  VillaMaria: 80
  VillaMercedes: 90
  VillaStaRosa: 50

Ciudad de destino: Salta

W hacia Salta: 70
Precio mínimo: 33
Camino: CiudadBs.As. -> Rosario -> Rafaela -> S.delEstero -> SanMiguelTucuman -> Salta
```

```
Peso máximo desde CiudadBs.As.:
  BellaVista: 50
  CiudadCordoba: 50
  Frias: 50
  Gral.Guemes: 70
  LaRioja: 70
  Laboulage: 90
  MarcosJuarez: 80
  Mendoza: 80
  MinaClavero: 50
  Rafaela: 70
  RioCuarto: 90
  Rosario: 90
  RosarioFrontera: 70
  Rufino: 100
  S.F.ValleCatamarca: 70
  S.delEstero: 70
  Salta: 70
  SanLuisCap: 80
  SanMiguelTucuman: 70
  SanSalvadorJujuy: 70
  Simoca: 50
  VillaMaria: 80
  VillaMercedes: 90
  VillaStaRosa: 50

Ciudad de destino: Mendoza

W hacia Mendoza: 80
Precio mínimo: 29
Camino: CiudadBs.As. -> Rufino -> Laboulage -> VillaMercedes -> SanLuisCap -> Mendoza
```

4. Análisis de resultados específicos / Discusión

4.1. Proyecto 1: Complejidad temporal en la Sala de Emergencias

El uso de un montículo binario como estructura subyacente de la cola de prioridad garantiza un rendimiento acotado para las operaciones críticas del triaje, evitando el costo lineal O(n) característico de las listas desordenadas o de las listas ordenadas por inserción:

- **Inserción (`insertar`):** O(log n) en el peor caso. El elemento se agrega al final del arreglo y `infiltArriba` lo intercambia con su padre, como máximo hasta la raíz. Como el montículo es un árbol binario completo, su altura es $\log_2 n + 1$.
- **Extracción del más urgente (`eliminar_min`):** O(log n) en el peor caso. Se extrae la raíz, se reubica el último elemento en su lugar y `infiltAbajo` (con `hijoMin`) lo baja como máximo hasta una hoja, recorriendo la altura del árbol.
- **Consulta del próximo paciente:** O(1), ya que el elemento de mayor prioridad (menor nivel de riesgo) siempre está en la raíz, en el índice 1 del arreglo.

Frente a una lista simple, donde buscar al paciente más crítico obliga a recorrer todos los nodos (O(n)), el montículo mantiene ambas operaciones en O(log n) aunque aumente el flujo de ingreso de pacientes.

4.2. Proyecto 2: Análisis de complejidad Big-O para Temperaturas_DB (Árbol AVL)

Como el árbol AVL se mantiene autobalanceado mediante rotaciones simples y dobles tras cada inserción o eliminación, su altura es O(log n). La siguiente tabla detalla la complejidad temporal de los métodos de la base de datos, donde n es la cantidad de mediciones almacenadas y k la cantidad de mediciones dentro del rango consultado:

Método de la interfaz	Complejidad Temporal (O)	Explicación del análisis
guardar_temperatura(temp, fecha)	$O(\log n)$	Busca la posición por fecha en el árbol AVL y, si es necesario, aplica hasta dos rotaciones para reequilibrar las alturas en el camino de vuelta.
devolver_temperatura(fecha)	$O(\log n)$	Realiza una búsqueda binaria estándar sobre la estructura jerárquica guiada por el orden cronológico de las fechas.
max_temp_rango(fecha1, fecha2)	$O(\log n + k)$	Utiliza un recorrido inorden con poda que visita únicamente los nodos que intersectan el intervalo dado (donde k es la cantidad de elementos devueltos).

min_temp_rango(fecha1, fecha2)	$O(\log n + k)$	Idéntico al anterior; explota la propiedad de orden del árbol para evitar recorrer subárboles completos fuera del rango.
temp_extremos_rango(fecha1, fecha2)	$O(\log n + k)$	Ejecuta el filtrado por rangos una única vez y extrae de forma lineal sobre los k elementos resultantes los valores extremos.
borrar_temperatura(fecha)	$O(\log n)$	Busca el nodo a eliminar, reestructura los enlaces y desencadena las verificaciones de balanceo y rotaciones hacia la raíz.
devolver_temperaturas(fecha1, fecha2)	$O(\log n + k)$	Recorre de forma acotada y ordenada el rango de fechas solicitado, construyendo la lista de salida sin recorrer toda la base de datos.
cantidad_muestras()	$O(1)$	Retorna una variable contadora interna actualizada dinámicamente en cada inserción o borrado exitoso.

4.3. Proyecto 3: Discusión de resultados en el Servicio de Transporte

El análisis conjunto de los resultados permite ver cómo interactúan las dos variantes de Dijkstra implementadas:

1. **Naturaleza complementaria de los algoritmos:** el primer ítem (máximo cuello de botella) maximiza la capacidad admisible mediante una cola de prioridad de máximos. El valor obtenido hacia cada destino (por ejemplo, W = 80 kg para Mendoza o W = 70 kg para Salta) es el peso máximo que se puede transportar sin superar la capacidad de ninguna ruta del camino. Ese mismo valor es el dato de entrada del segundo ítem.
2. **Impacto del filtrado en el costo monetario:** el cálculo del precio mínimo requiere filtrar previamente el grafo, descartando las rutas cuya capacidad sea inferior al peso W que se desea transportar. Esto se comprobó con el destino Mendoza: sin el filtro, el camino más barato pasa por Río Cuarto con un costo de 27, pero es inviable porque la ruta Río Cuarto → San Luis admite solo 60 kg. Al aplicar el filtro por W, el algoritmo selecciona el camino válido más barato, con un costo de 29.
3. **Complejidad temporal:** sea V la cantidad de ciudades y E la cantidad de rutas. Ambos algoritmos hacen V extracciones de costo O(log V) y hasta E actualizaciones de clave. Como `decrementar_clave` y `aumentar_clave` buscan el elemento recorriendo todo el arreglo, cada actualización cuesta O(V), y el costo total es O(V log V + E·V) = O(E·V). La teoría indica O((V + E) log V) para Dijkstra con montículo binario, pero ese valor supone que actualizar una clave cuesta O(log V). Se podría alcanzar guardando un diccionario que relacione cada elemento con su posición en el montículo. En la red del archivo (V = 25 y E = 35 rutas distintas) la diferencia no se nota en el tiempo de ejecución.

5. Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas:

- **Gemini**
- **Claude ai**
- **GitHub Copilot**

Descripción del uso:

- **Generación de ideas y estructura:** Se utilizó **Gemini** para obtener perspectivas alternativas en el planteo conceptual de los problemas y estructurar el enfoque inicial de resolución.
- **Asistencia en código:** Se emplearon **Claude** y **GitHub Copilot** para la revisión, validación lógica y optimización de las funciones y algoritmos implementados en el proyecto.

- **Consulta teórica:** Se consultó a **Claude** como apoyo conceptual para resolver dudas teóricas y evaluar el orden de eficiencia (O) de diversas estructuras de datos.
- **Asistencia en redacción:** Se utilizó **Gemini** para la revisión estilística, corrección ortotipográfica y mejora de la fluidez en la redacción final de este informe.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

6. Trabajo en equipo

6.1. Organización y comunicación

El equipo coordinó el desarrollo del trabajo a través de **WhatsApp** utilizado para resolver dudas puntuales durante la semana (sobre todo tras las clases teóricas de los martes) y un **repositorio en GitHub** como eje para la integración del código. Las jornadas principales de desarrollo conjunto y puesta en común se llevaron a cabo los viernes durante las clases prácticas presenciales con el docente Juan Rizzato, completando las tareas pendientes de forma remota durante los fines de semana.

6.2. Mecanismos de coordinación y resolución de discrepancias

Ante dudas teóricas o de diseño en el código, el procedimiento consistió en consultar a herramientas de Inteligencia Artificial como guía inicial de exploración y, posteriormente, validar los criterios técnicos directamente con el docente en las clases prácticas.

6.3. Estimación inicial y tiempo efectivamente dedicado

Estimación inicial

Se calculó un total estimado de **28 horas de trabajo**, distribuidas de la siguiente manera:

- **Búsqueda de información y análisis teórico:** 4 horas.
- **Desarrollo de código (Proyectos 1, 2 y 3):** 20 horas.
- **Pruebas unitarias y depuración de errores:** 2 horas.
- **Redacción del informe y revisión final:** 2 horas.

7. Conclusiones

Cumplimiento de objetivos: se implementaron y probaron los tres proyectos: una cola de prioridad genérica sobre montículo binario para la sala de emergencias, una base de datos de temperaturas sobre un árbol AVL y un servicio de transporte resuelto con dos variantes de Dijkstra. En cada caso la estructura elegida responde a las operaciones más frecuentes del problema.

Dificultades y resolución:

- Pasar el montículo y la cola de prioridad de mínimos a máximos exigió invertir las comparaciones y, además, negar el contador de llegada para que los empates siguieran saliendo por orden de llegada.
- El Dijkstra del libro llamaba a métodos que la cola del equipo no tenía o tenía con otro nombre. Se resolvió agregando `construirMonticulo` y ajustando los nombres.
- Cada ruta tiene dos datos y `Vertice` guarda una sola ponderación por arista, por lo que se armó un grafo para las capacidades y otro para los precios.
- Al reutilizar la cola en Dijkstra apareció que `decrementar_clave` cuesta O(n), lo que aleja el costo total del valor teórico.

Aprendizajes: la estructura de datos se elige a partir de las operaciones críticas del problema, y el costo de una operación auxiliar, como la búsqueda dentro de `decrementar_clave`, puede determinar la complejidad de todo el algoritmo. También se vio por qué importa balancear el árbol: las muestras llegan ordenadas por fecha, y un árbol sin balancear degeneraría en una lista. Por último, reutilizar una misma estructura en varios proyectos mostró el valor de mantenerla genérica y separada de la aplicación.

8. Referencias y Bibliografía

- Miller, B., & Ranum, D. (2013). *Solución de problemas con algoritmos y estructuras de datos usando Python* (M. Orozco-Alzate, Trad.; 2.a ed.).
- Material teórico de la cátedra de Algoritmos y Estructuras de Datos: Unidad 1 (Estructuras de datos y tipos de datos abstractos), Unidad 2 (Diseño y análisis de algoritmos y clases de complejidad), Unidad 5 (Árboles y algoritmos asociados) y Unidad 6 (Grafos).
- Códigos iniciales para estudiantes de la cátedra: Proyecto 1 (Sala de Emergencias) y Proyecto 2 (Temperaturas_DB, archivo de muestras).
- Repositorio del equipo con el código de los tres proyectos.